

Assignment 3

Real-Time Streaming Analytics Dashboard

Industry: Ride-Hailing & Urban Mobility Technology

Technology Stack: Apache Kafka, Python, MySQL, Grafana

Name	Aman Verma
Roll No.	065067
Program	PGDM (BDA)
Institute	FORE School of Management
Date	1 October 2026

1. Project Objective

The objective of Assignment 3 is to extend the Kafka-based ride-hailing streaming pipeline developed in Assignments 1 and 2. The pipeline now consumes streaming data and stores it in a relational SQL database for analytics and visualization.

The implementation uses Apache Kafka as the streaming layer, a Python consumer to process Kafka messages, MySQL for persistent storage, and Grafana for interactive dashboards. The dashboard gives operational visibility into ride demand, driver supply, driver status and fare patterns.

Assignment 1 established why real-time processing of ride requests and driver GPS streams matters. Operational decisions such as driver matching, supply balancing and surge pricing depend on current data, not on delayed batch reports.

2. Background: Assignments 1 and 2

The project was developed step by step across three assignments.

Assignment 1

Assignment 1 identified Ride-Hailing & Urban Mobility Technology as the industry and designed a Kafka-based streaming architecture with these data sources:

- User and static master data
- Ride requests
- Driver GPS / location feed
- Trip status
- Traffic and weather data

The proposed design uses Kafka topics to ingest streaming events and a processing layer to calculate driver matching, ETA and zone-level surge pricing.

Assignment 2

Sample data was generated for four datasets:

Dataset	Records
user_static_master.json	20
ride_requests.json	30
driver_location_feed.json	40
trip_status.json	25
Total	115

Three Kafka topics were created: **user_static_master**, **ride_requests** and **driver_location_feed**. Each topic was configured with two partitions in the local Kafka setup. The Python producer used kafka-python and sent records with partition-aligned keys.

3. Assignment 3 Architecture

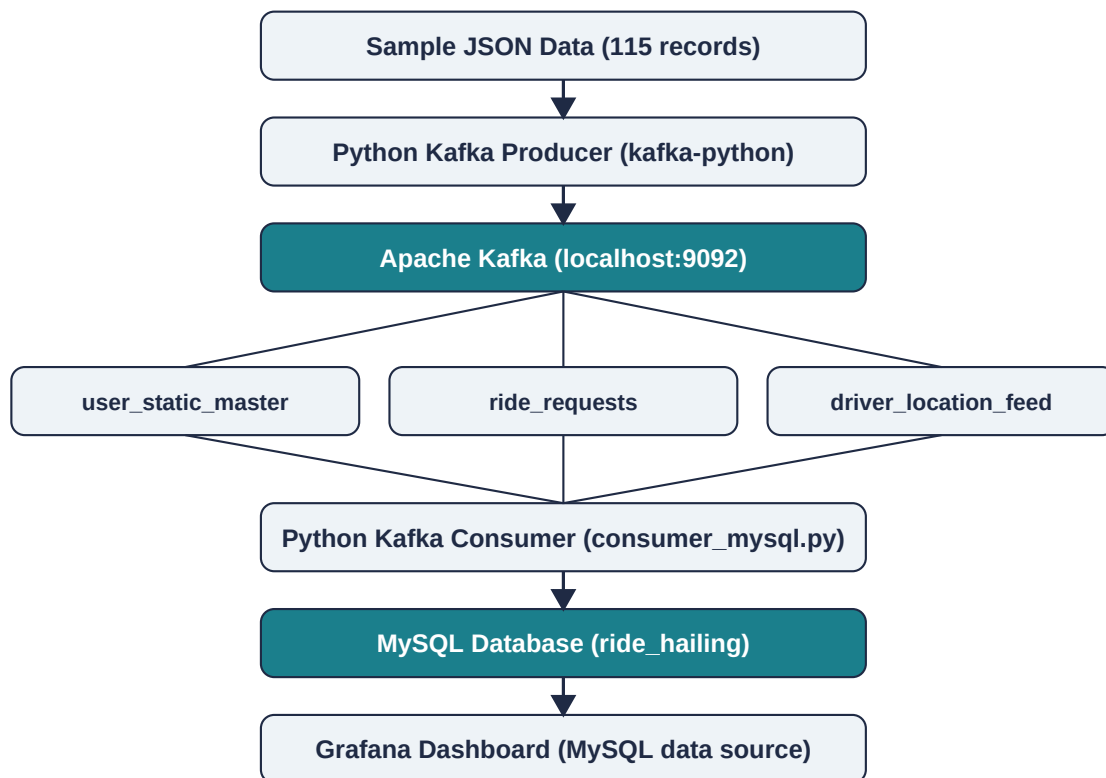


Figure A: End-to-end data flow for Assignment 3

Assignment 3 adds MySQL as the persistent analytical storage layer. The Python Kafka consumer subscribes to the three Kafka topics and inserts each consumed record into the matching MySQL table. Grafana connects directly to MySQL and uses SQL queries to build the KPI cards and charts.

The Docker environment (Kafka, Zookeeper and MongoDB containers) was running during execution, as shown below.

```
Run 'docker run --help' for more information
(venv) PS C:\SDA\Assignment 2> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
dc74d9e39dd2	mongo	"docker-entrypoint.s..."	2 months ago	Up 5 hours	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp	sda-mongo-1
1b99f91943e2	wurstmeister/kafka	"start-kafka.sh"	2 months ago	Up 4 hours	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp	sda-kafka-1
6b99d26944a6	wurstmeister/zookeeper	"/bin/sh -c '/usr/sb..."	2 months ago	Up 5 hours	0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp	sda-zookepe
r-1						

Figure 1: Docker containers running the Kafka and supporting infrastructure

4. Kafka Consumer Implementation

The consumer script `consumer_mysql.py` uses the `kafka-python` library to read messages from all three topics. It identifies each JSON message by its Kafka topic and inserts it into the matching MySQL table.

Kafka Topic	MySQL Table
user_static_master	user_static_master
ride_requests	ride_requests
driver_location_feed	driver_location_feed

The consumer also stores Kafka metadata with every record: **topic**, **partition** and **offset**. This gives traceability between the streaming events and the rows in the database.

```
(venv) PS C:\SDA\Assignment 2> python consumer_mysql.py
MySQL connection successful
C:\SDA\Assignment 2\consumer_mysql.py:63: DeprecationWarning: value_deserializer does not implement kafka.serializer.Deserializer
consumer = KafkaConsumer(
-----
Kafka    : localhost:9092
Topics   :
  • user_static_master
  • ride_requests
  • driver_location_feed
MySQL    : ride_hailing

Website                                     Ln 67, Col 41  Spaces: 4  UTF-8  CRLF  {} Python  venv (3.1
```

Figure 2: Consumer connected to Kafka (localhost:9092), subscribed to the three topics, and connected to MySQL database ride_hailing

The deprecation warning shown in the output relates to how the value deserializer is declared in kafka-python. It does not affect the consumer's operation.

5. Grafana Dashboard

The final dashboard has four KPI cards and four analytical visualizations, all driven by SQL queries against the MySQL database.

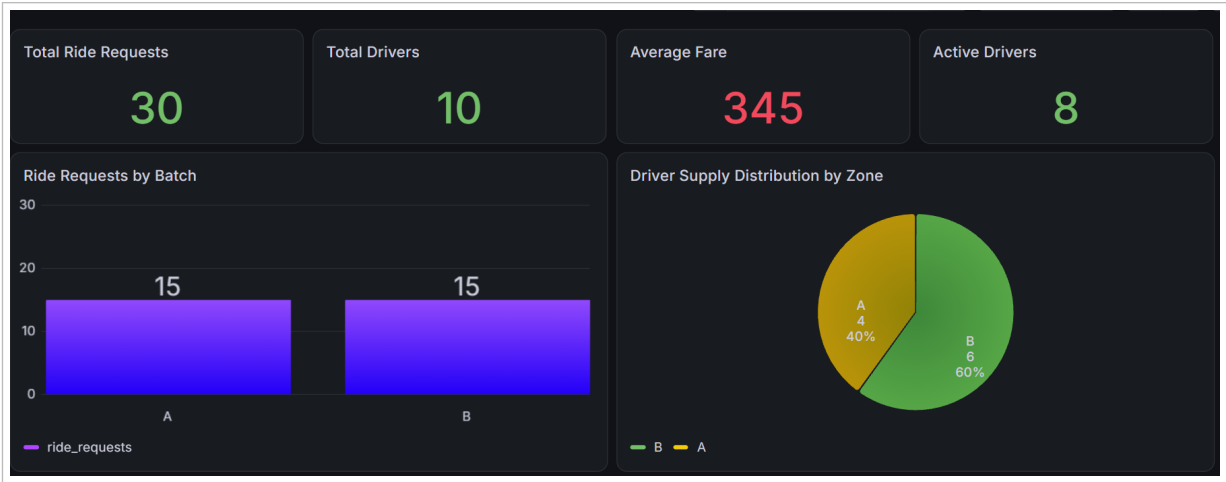


Figure 3: Ride-Hailing Real-Time Analytics Dashboard in Grafana (KPI cards, ride requests by batch, driver supply by zone)

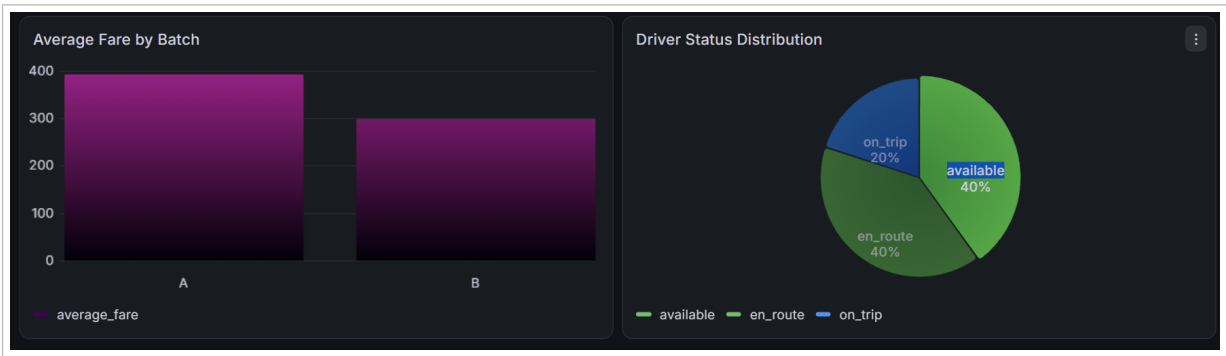


Figure 4: Average fare by batch and current driver status distribution

KPI Cards

KPI	Value
Total Ride Requests	30
Total Drivers	10
Average Fare	Rs. 345
Active Drivers	8

Visualizations

- 1. Ride Requests by Batch.** Batch A has 15 requests and Batch B has 15 requests. The generated ride requests are split evenly between the two batches.
- 2. Driver Supply Distribution by Zone.** The latest driver-location records show 4 drivers in Zone A (40%) and 6 drivers in Zone B (60%).

3. Average Fare by Batch. Batch A has a visibly higher average estimated fare than Batch B. From the bar heights, Batch A is roughly Rs. 390 and Batch B roughly Rs. 300, which is consistent with the overall average fare of Rs. 345 shown on the KPI card.

4. Driver Status Distribution. Based on each driver's latest record: Available 40%, En-route 40%, On-trip 20%. So 80% of drivers are either available or en-route.

SQL-Based Analysis

Each panel uses a SQL query on the MySQL data source. The driver-status panel, for example, uses the ROW_NUMBER() window function, partitioned by driver and ordered by latest record first. This keeps only each driver's most recent record, so every driver is counted once and the chart reflects current state, not the full history of location events.

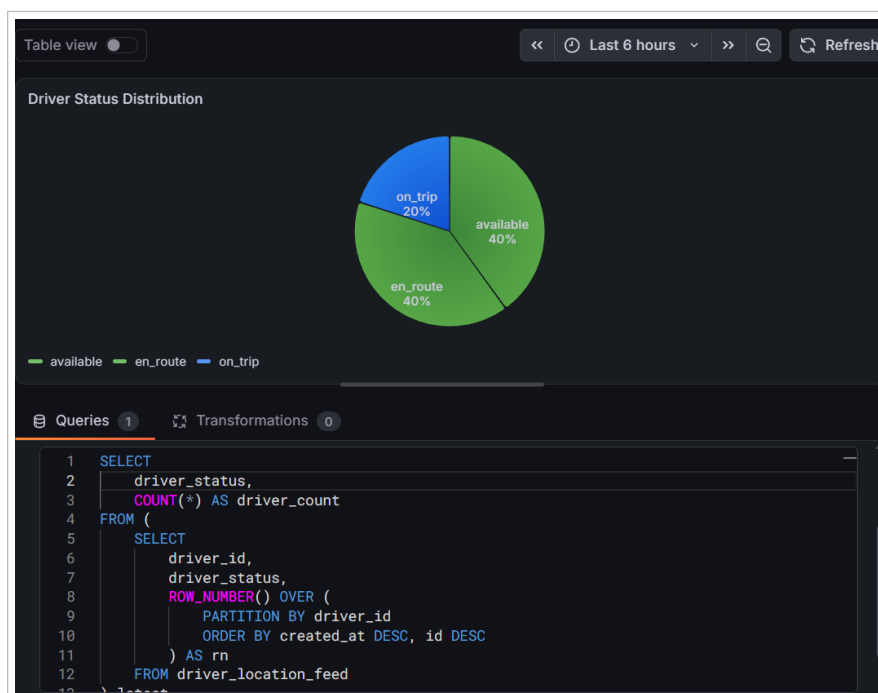


Figure 5: SQL-based latest-status analysis used to generate the Grafana driver status visualization

6. Business Insights

Insight 1: Balanced ride demand

The 30 ride requests are divided evenly between Batch A and Batch B (15 each). There is no demand skew between the batches in this dataset.

Insight 2: Uneven driver distribution

The latest driver-location data places 6 drivers in Zone B and 4 in Zone A. Driver availability is not balanced geographically, and Zone A has fewer drivers to serve its demand.

Insight 3: Driver availability

The latest status split is 40% available, 40% en-route and 20% on-trip. This means 80% of drivers are not currently on a trip, and the Active Drivers KPI of 8 out of 10 matches this.

Insight 4: Fare difference between batches

Batch A has a noticeably higher average fare than Batch B even though both have the same number of requests. This suggests the trips in Batch A are longer or priced higher. The dataset alone cannot confirm the cause.

Insight 5: Supply-demand monitoring

The dashboard brings together demand data from **ride_requests** and supply data from **driver_location_feed**. Operations teams can use it to spot zones that may face a shortage of drivers.

Insight 6: Potential operational actions

Assignment 1 named real-time supply-demand imbalance and surge activation as the key business decision the streaming architecture should support. In the current dashboard, zone-level supply and demand could feed future actions such as:

- Driver repositioning
- Incentives for drivers
- Surge pricing
- Supply shortage alerts

Note: The current system visualizes this data. It does not automatically activate surge pricing or run an automated decision engine.

7. Technical Challenges and Solutions

Challenge	Issue	Solution
Kafka connectivity	Broker connectivity first had to be verified.	Kafka Docker containers were started, and the producer and consumer connected through localhost:9092.
MySQL setup	The ride_hailing database and tables had to exist before the consumer could save messages.	A Python MySQL connector script created the database and tables.
Duplicate Kafka consumption	Consumer groups remember their offsets, so already-read messages are not read again.	A new consumer group was used during testing so earlier messages could be consumed again.
Grafana time range	A time-series query showed "Data outside time range" because the sample records were inserted in one batch, not spread over time.	The dashboard was redesigned around categories and current-state analysis instead of presenting sample data as a continuous time series.

The last challenge shows the difference between streaming ingestion and historical visualization: data that arrives in a short burst does not make a meaningful time series.

8. Limitations

- The implementation uses a generated sample dataset, not a production ride-hailing stream. It has 30 ride requests and 40 driver-location events, so it does not represent real city-scale traffic.

- The dashboard focuses on visualization and monitoring. Automated driver matching, ETA calculation, surge activation and traffic/weather integration from Assignment 1 are not implemented in this assignment.
- The consumer ingests three of the four Assignment 2 datasets. The trip_status dataset is not loaded into MySQL or shown on the dashboard.

9. Conclusion

Assignment 3 extends the Kafka-based ride-hailing streaming pipeline with MySQL as a persistent SQL storage layer and Grafana as the visualization layer. The pipeline shows the full flow from Kafka message ingestion to relational storage and dashboard analysis.

The dashboard gives visibility into ride demand, driver supply, driver status and fare patterns. It shows how streaming analytics can support operational decisions in ride-hailing, particularly around spotting supply-demand imbalances and monitoring driver availability.